

Building Agents with OpenAI's AgentKit and Responses API

Lucas Soares

O'Reilly Live Training

September 2026

Table of Contents

1. The Problem: Why Layers Matter
2. Layer 1: Responses API — the primitive
3. Layer 2: Agents SDK — the framework
4. Layer 3: AgentKit — the product platform
5. Agent Builder — visual workflow orchestration
6. When to Use Each Layer

The Problem

Building Production Agents Is Hard

An API call gets you started. But production requires much more:

- **Orchestration** — chaining multiple LLM calls + tool executions
- **Multi-agent routing** — delegating tasks to specialized agents
- **Guardrails** — input/output validation, safety checks
- **Observability** — tracing, debugging, cost monitoring
- **User Interface** — a chat experience your users actually interact with

OpenAI's answer: a layered stack — each layer builds on the previous one.

The Three-Layer Stack



Each layer builds on the previous — you pick the level of abstraction you need.

Layer 1: Responses API

The Primitive

Responses API — What & Why

Released **March 2025**. Replaces both Chat Completions API and Assistants API.

Key Innovations

- **Items-based I/O** — polymorphic output (text, tool calls, reasoning)
- **Server-side state** via `previous_response_id`
- **Reasoning preservation** across turns

Built-in Tools

- `web_search`
- `file_search`
- `code_interpreter`
- `computer_use`
- `mcp` (Model Context Protocol)

This is what we've been using in the notebooks so far.

^[1]Why we built the Responses API

Layer 2: Agents SDK

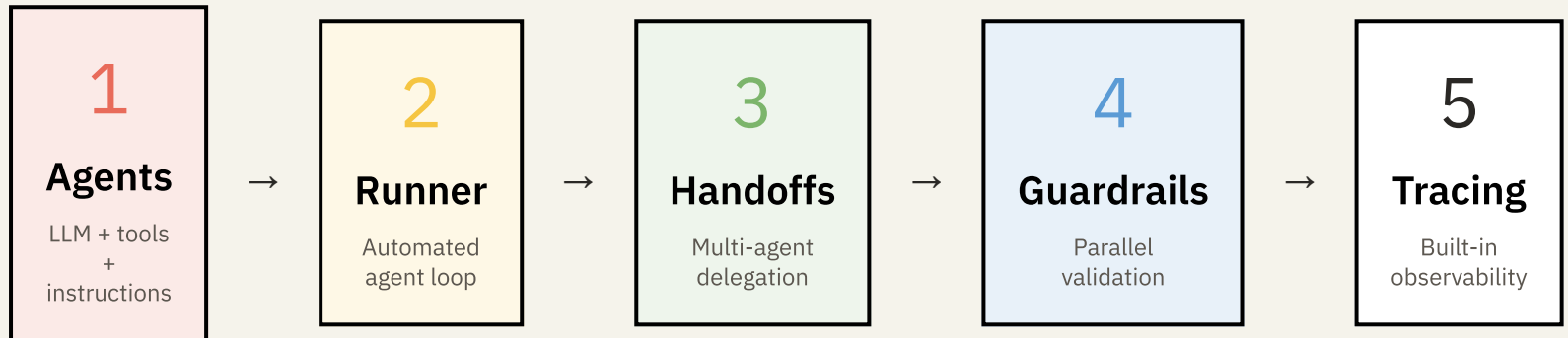
The Framework

Agents SDK — Overview

Open-source framework. **Python + TypeScript**.

```
pip install openai-agents
```

The 5 Core Primitives



Works with **100+ LLMs** — not locked to OpenAI.

[1] [openai-agents-python \(GitHub\)](#).

[2] [Agents SDK Documentation](#)

Layer 3: AgentKit

The Product Platform

AgentKit — The Full Platform

Launched at **DevDay, October 2025**. No platform fee — standard API pricing.



Agent Builder

Visual drag-and-drop canvas for composing multi-agent workflows



ChatKit

Embeddable chat UI — React components + Web Component



Connector Registry

Pre-built data connectors — Dropbox, Google Drive, SharePoint, MCP



Evals for Agents

Datasets, trace grading, automated prompt optimization

Agent Builder

Visual Workflow Orchestration

Agent Builder — Deprecation Notice



Agent Builder & Evals are being shut down

OpenAI will retire **Agent Builder** and **Evals** on **November 30, 2026**.

The demos in this module will not be reproducible after that date.

The patterns you are learning — multi-agent orchestration, handoffs, guardrails, classifiers, and retrieval — are the **fundamental building blocks**. They are not tied to this UI. When the tooling changes, the thinking stays.

Source: community.openai.com/t/deprecation-notice-agent-builder/1382650

Where the patterns live after November 2026

AgentKit is a bundle, not one product. On November 30, **Agent Builder** and **Evals** retire. **ChatKit** continues, self-hosted only. The **Agents SDK** continues unchanged.

Developers → Agents SDK

```
pip install openai-agents
```

- Same patterns in Python code
- Handoffs, guardrails, file search as classes
- Full developer control, self-hosted

No-code → Workspace Agents

ChatGPT Business / Enterprise

- No-code, connects to Slack / Google Drive / Salesforce
- Internal team automation — not a developer API

ChatKit is not killed — only its hosted mode is. The OpenAI-hosted “Recommended integration” used Agent Builder as its backend, so it retires with it on November 30. The “Advanced integration” stays: the same widget on **your own backend** — ChatKit Python SDK + Agents SDK. Module 4's app still ships; you now run the server.

Module rewrite targeting Agents SDK is planned for a future course revision.

Same patterns, outside OpenAI

Agent Builder was one way to *draw* the flow. The flow itself carries over to any orchestration tool:

Guardrails → Classifier → specialist agents → human approval before any action

n8n

Visual — closest to the canvas

- Drag-and-drop workflows with AI agents
- Self-hostable
- OpenAI, Anthropic, Google models

LangGraph

Code — a graph of steps + shared state

- Long-running, stateful agents
- Built-in human-in-the-loop
- Works with any model

CrewAI

Code — teams of role-based agents

- Agents defined by role, goal, backstory
- Flows for routing and state
- Anthropic, Gemini, Ollama, and more

The code options lose what the canvas gave for free: a diagram a non-engineer can read and approve. n8n keeps it.

Agent Builder — What It Is

A **visual drag-and-drop canvas** for composing multi-step agent workflows — no orchestration glue code required.

Part of AgentKit

- Sits alongside **ChatKit**, **Connector Registry**, and **Evals**
- Powered by the **Responses API** under the hood
- Standard API pricing — no platform fee

What You Get

- Build multi-agent workflows visually
- Test interactively before deploying
- Publish and version with one click
- Connect to ChatKit or export as code

Target users: developers who want visual orchestration *and* teams who need to ship quickly without writing agent loop code.

Core Concepts — Workflows & Nodes

Workflow

The blueprint. The complete graph of nodes that defines how a request is processed.

- Versioned and publishable
- Has a unique, stable ID
- The unit you build, test, and deploy
- Can be re-used across ChatKit, SDK, or API

Node

The building block. Each node does exactly one thing.

- Typed inputs and outputs
- Connections enforce data contracts
- Executes independently — traceable
- Chain nodes to compose complex behavior

A workflow is the graph. A node is the vertex. Edges carry typed data.

Node Types

Start

Entry point — accepts initial user input

Agent

Calls an LLM with instructions + tools

Classifier

Routes to one of N branches based on output category

If / Else

Conditional logic on data values

File Search

Vector search over uploaded documents (RAG)

Transform

Maps data between incompatible node outputs/inputs

MCP

Calls a hosted MCP server tool (third-party data/actions)

Guardrails

Input/output safety validation before/after agents

End

Returns final output to the caller

Typed connections — each node declares its output schema; the canvas enforces that downstream nodes receive what they expect.

Templates vs Blank Canvas

Start from a Template

Best when: your use case fits a known pattern

- Support triage with routing
- Document Q&A (RAG)
- Research assistant
- Multi-step approval workflow

Pre-wired nodes — modify instructions, swap models, go.

Start Blank

Best when: designing something novel

- Custom multi-agent topology
- Unique data transformation pipeline
- Experimental classifier routing
- Prototyping from first principles

Add nodes one by one — full control over the graph.

Preview & Testing

Agent Builder has a built-in interactive test panel — no deploy step required.



Interactive Runs

Send test inputs and see full output in real time



File Attachments

Attach PDFs and documents for File Search workflows



Per-Node Traces

Inspect what each node received and returned

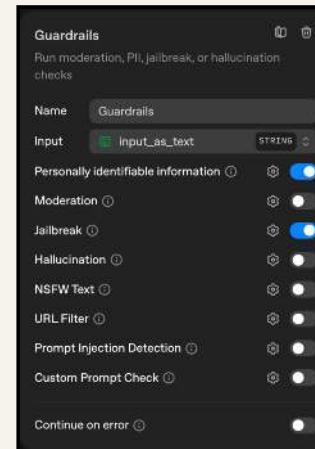
Demo 1 (Course Assistant) — the simplest test case: Start → Agent → End. Per-node trace shows exactly what the agent saw and returned.

Guardrails — Open-Source Safety Layer

A **modular, open-source safety layer** that runs before and after your agents — no changes to agent logic required.

What It Protects Against

- PII detection — Block or Redact mode
- Jailbreak attempts
- Prompt injection attacks
- Off-topic requests
- Sensitive data leakage in responses



Demo 2 (Support Triage) — Guardrails node gates every request before the Classifier. PII set to Block mode; jailbreak detection enabled.

MCP Integration

MCP nodes connect your workflow to hosted third-party tools via the Model Context Protocol.

How It Works

- Add an MCP node to any workflow
- Pick a server from the Connector Registry
- The agent gains access to that tool set
- Examples: Slack, GitHub, Google Drive, custom APIs

⚠ Context-Window Cost

Each MCP tool you add expands the model's context window with tool definitions.

Stack wisely: more tools = higher latency & cost per call.

Demo 3 (File Search RAG) uses the built-in File Search node — same composition pattern as MCP, but backed by a vector store instead of an external API.

Widget Studio

Go beyond plain text — render **rich, structured outputs** directly from agent responses.



Tables & Cards

Structured data rendered for review



Forms

Approval and input UIs from agent output



Custom Layouts

Enterprise dashboards, data review UIs

Widget Studio is configured in the workflow — the agent outputs structured JSON; Widget Studio renders it as a component in ChatKit.

Versioning & Publishing

Workflows are **first-class versioned objects** — not just scripts you save and overwrite.

Each Version Has

- A unique, stable workflow ID
- A published deployment URL / API endpoint
- An immutable snapshot of the graph

Workflow Lifecycle

- Prior versions remain accessible
- Safe rollbacks with no downtime
- Staged promotion (dev → staging → prod)
- Point ChatKit at a specific version ID

Deployment Paths

Once your workflow is published, two paths to ship it:



ChatKit

(recommended default)

- Embed a chat UI in minutes
- Handles auth, streaming, file upload
- No backend agent loop code needed
- Point at a workflow ID — done



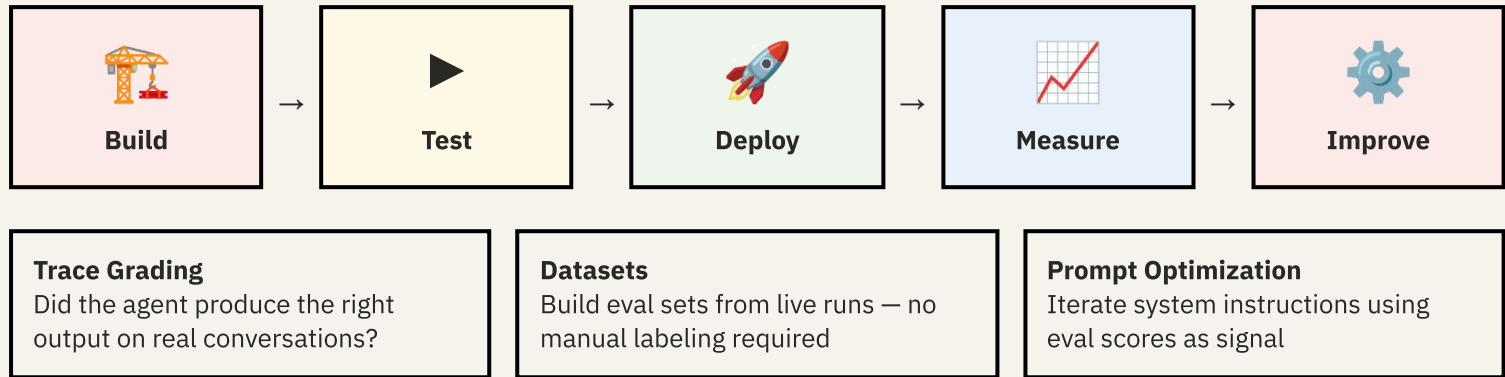
SDK Code Export

- Export as Python or TypeScript
- Full server-side orchestration control
- Custom approval flows & state management
- Tool execution hooks & multi-runtime

Start with ChatKit. Export to SDK only when you need capabilities the hosted path can't provide.

Evals Integration

Evals closes the loop between building and improving your workflow.



Agent Builder vs Agents SDK

Reach for Agent Builder when...

- Visual builder for non-dev teammates
- Shipping to ChatKit is the goal
- A template is close to your use case
- You want Evals + iteration in the UI
- Prototyping and testing workflows fast

Reach for Agents SDK when...

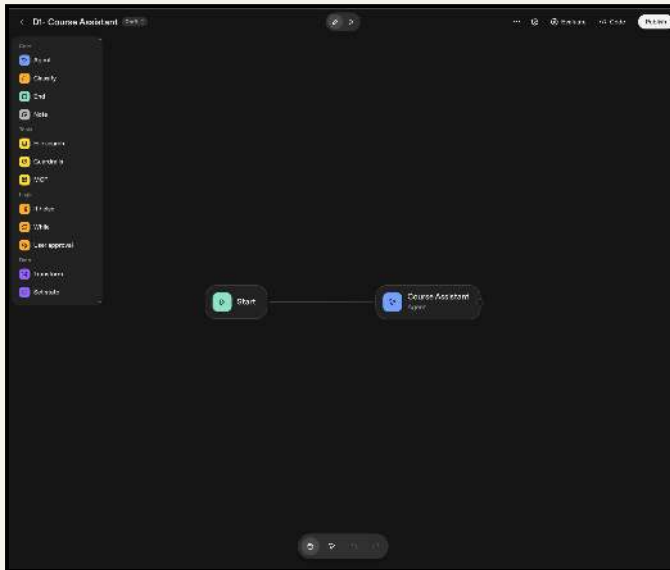
- Server-side orchestration & custom state
- Custom human-in-the-loop approval flows
- Fine-grained tool execution control
- Multi-runtime or non-OpenAI deployment
- Full code ownership is a requirement

These paths are **not mutually exclusive** — prototype in Agent Builder, export to SDK for production hardening.

Demo: Module 3 — Agent Builder Use Cases

Demo 1 – Course Assistant

Simplest workflow: Start → Agent → End



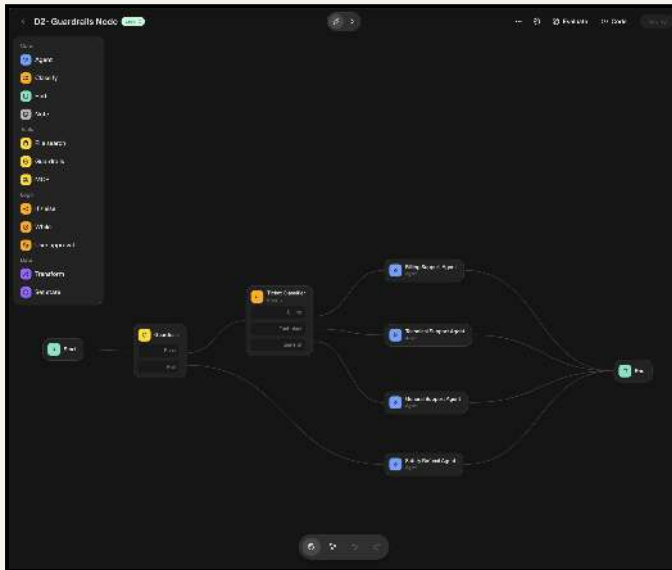
What It Demonstrates

- Minimal node composition — one Agent node
- How to set system instructions on a node
- Per-node trace inspection in preview mode
- Good starter template for any assistant

Model: gpt-5.6-luna with chat history enabled

Demo 2 — Support Triage with Guardrails

Multi-agent routing + safety: Guardrails → Classifier → 3 specialists (or Safety Refusal)

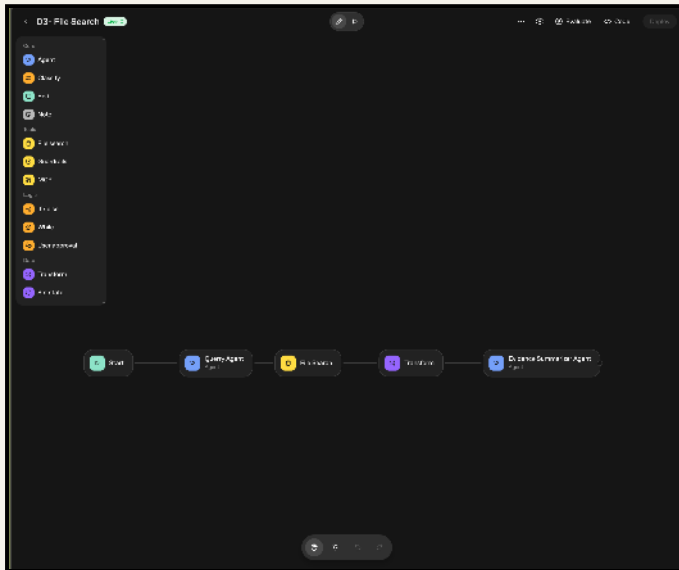


What It Demonstrates

- Guardrails node: PII block mode + jailbreak detection
- Classifier routing to Billing / Technical / General
- Safety Refusal Agent for policy violations
- Confidence-based routing prevents unsafe paths

Demo 3 — File Search RAG (SEC 10-K Filings)

Grounded answers from documents: Start → Query Agent → File Search → Transform → Evidence Summarizer



What It Demonstrates

- File Search node over 10 SEC 10-K filings
- Transform node: bridges typed output mismatch
- Evidence Summarizer with cited sources
- Vector store with real financial documents

Query: "What does NVIDIA say about AI data centers and supply chain risks?"

ChatKit — Embeddable Chat UI

Frontend-only — React components + framework-agnostic Web Component.

```
import { ChatKit, useChatKit } from '@openai/chatkit-react';

const chatkit = useChatKit({
  api: { getClientSecret },
  ....
});

return <ChatKit control={chatkit.control} />;
```

Includes: file upload, light/dark theming, tool invocation display, chain-of-thought visualization, suggestion chips, event system, and more.

[1][ChatKit — OpenAI Docs](#)

[2][chatkit-js \(GitHub\)](#)

ChatKit — Backend Token Exchange

```
// POST /api/create-session (your backend)
const response = await fetch(
  "https://api.openai.com/v1/chatkit/sessions",
  {
    method: "POST",
    headers: {
      ....
    }
  });

const { client_secret } = await response.json();
// Return client_secret to the frontend
```

Key pattern: Your server-side API key is exchanged for a short-lived client token. Per-user sessions via cookies. The frontend *never* sees your API key.

Demo: ChatKit App

When to Use Each Layer

Direct API Control

- **Responses API** — direct calls, full control, learning

Multi-Agent Systems

- **Agents SDK** — orchestration, custom logic, guardrails

Ship a Chat UI

- **ChatKit** — embed a chat experience fast

Visual Workflows & Evals

- **Agent Builder** — drag-and-drop, non-dev teams
- **Evals** — measure and improve agent quality

Key Takeaways

- **Start with Responses API** for maximum control and the deepest learning
- **Add Agents SDK** when you need multi-agent orchestration, guardrails, tracing
- **Use AgentKit / ChatKit** when you're ready to ship to real users
- Each layer **builds on the previous** — no vendor lock-in at the SDK level
- The whole stack is available at **standard API pricing** — no platform fee

What We Built

From notebooks to full-stack agent apps across three modules:

- **Paper Chat App** — interactive Q&A on uploaded PDFs (Module 1)
- **Video Script Generator** — paper → 60-second video script with DALL-E scenes (Module 1)
- **Research Report App** — structured analysis exported as Markdown (Module 1)
- **3 Agent Builder Workflows** — Course Assistant, Support Triage, File Search RAG (Module 3)

The patterns you learn here apply at every layer of the stack.

Q&A & References

- [Why we built the Responses API](#) — OpenAI Blog
- [Responses API Reference](#) — OpenAI Docs
- [openai-agents-python](#) — GitHub
- [Agents SDK Documentation](#) — OpenAI
- [Introducing AgentKit](#) — OpenAI
- [ChatKit Guide](#) — OpenAI Docs
- [chatkit-js](#) — GitHub
- [Agent Builder Guide](#) — OpenAI Docs
- [Guardrails \(open-source\)](#) — GitHub

Questions before we move on?

